

ROBOTICS & EMBEDDED SYSTEMS

8-Bit Processor Design and Validation

Digital Logic, Microprocessor Architecture, and System Testing

Project Evidence Report

Timothy R. Williams | RBT Student | August 2026

Project Overview

For this project, I designed and tested an 8-bit processor in Logisim Evolution. I did not start with one complete CPU block. I built the smaller digital-logic circuits first, verified each one, and then connected them into a working processor. The finished system includes arithmetic, registers, instruction and data memory, a program counter, instruction decoding, control signals, an accumulator, status flags, and an output register.

My final test program loads values, stores data in RAM, performs an addition, sends the answer to the output register, and halts. The expected answer is 8, and the final screenshots show that result remaining stable after another clock pulse.

Demonstration Program

Address	Code	Instruction	Purpose
0	15	LDI 5	Load the value 5 into the accumulator.
1	3E	STA E	Store 5 in data-memory address E.
2	13	LDI 3	Load the value 3 into the accumulator.
3	4E	ADD E	Add the stored 5 to the current value 3.
4	80	OPT	Copy the result 8 to the output register.
5	F0	HLT	Stop the program counter and hold the result.

Objective Alignment

Primary alignment: RBT Objective 2 - embedded microprocessor system skills and digital logic design.

This project is strong evidence for that objective area because I designed the processor from digital-logic building blocks, created and connected the datapath and control path, established an instruction format, used ROM and RAM, and validated the system through repeatable tests. The work goes beyond drawing a block diagram because the completed circuit executes a stored program one clock pulse at a time.

Cri	Evidence	Status
Digital logic design	Full adders, ripple-carry addition, two's-complement subtraction, Boolean logic, multiplexers, comparators, registers, and status flags.	Demonstrated
Microprocessor architecture	Program counter, instruction memory, decoder, control unit, ALU, accumulator, data memory, output register, buses, and clock/reset controls.	Demonstrated
System integration	Separately tested modules were connected into a single CPU datapath and control system.	Demonstrated
Testing and troubleshooting	Normal, overflow, negative, zero, hold, memory, halt, and complete-program tests were documented.	Demonstrated
Physical implementation	Logisim design provides the functional foundation for later Basys 3 FPGA implementation.	Next phase

Notes: *If I had more time and funds, I would have purchased a Basys 3 and advanced this project beyond the simulation.*

Step 1: Building and Testing the 8-Bit Ripple-Carry Adder

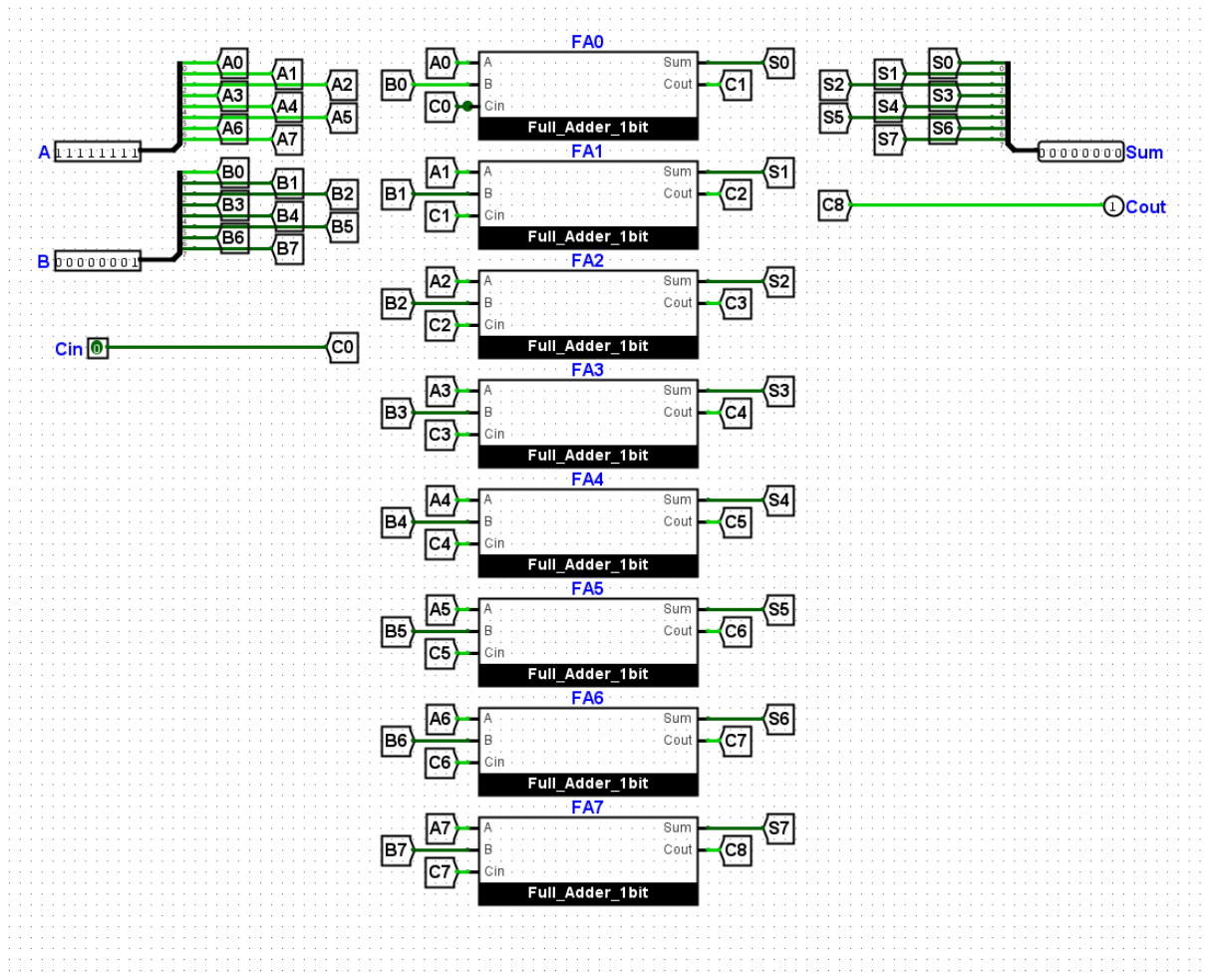


Figure 1. Building and testing the 8-Bit Ripple-Carry Adder

What I built or tested. I built the 8-bit adder from eight 1-bit full-adder blocks. Each carry output feeds the next stage, which lets the circuit add two full 8-bit numbers instead of treating each bit separately.

What this proves. For this test I used 11111111 plus 00000001 with carry-in set to 0. The 8-bit sum rolled over to 00000000 and Cout became 1. That is the expected overflow behavior and confirms that the carry chain works through all eight stages.

Step 2: Testing Normal Addition in the Add/Subtract Circuit

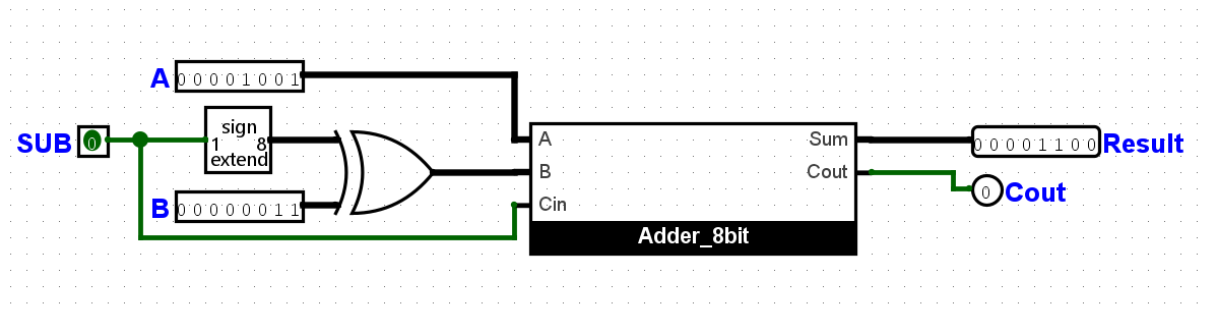


Figure 2. Testing Normal Addition in the Add/Subtract Circuit

What I built or tested. I added an XOR stage and a SUB control so the same adder could handle addition and subtraction. With SUB set to 0, the B input passes through unchanged, and the circuit behaves like a normal 8-bit adder.

What this proves. The test adds 00001001 (9) and 00000011 (3). The result is 00001100 (12), confirming that normal addition still works after the subtraction controls were added.

Step 3: Testing a Negative Two's-Complement Result

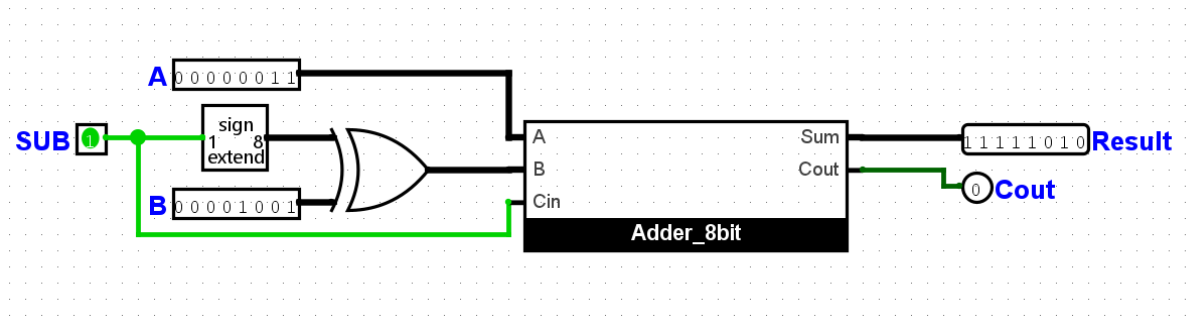


Figure 3. Testing a Negative Two's-Complement Result

What I built or tested. For subtraction, SUB is sent to every XOR gate on the B input and also becomes the initial carry-in. This performs $A + \text{NOT } B + 1$, which is the standard two's-complement subtraction method.

What this proves. The circuit subtracts 9 from 3. The result is 11111010, which represents -6 in 8-bit two's-complement form. This confirms that the subtract path can produce and represent a negative answer.

Step 4: Verifying the Zero Flag

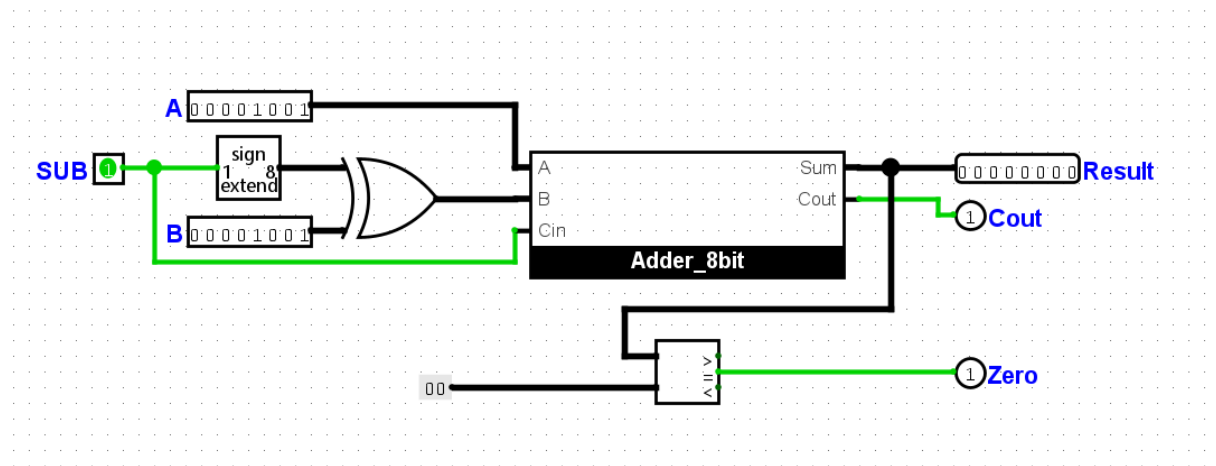


Figure 4. Verifying the Zero Flag

What I built or tested. I connected the arithmetic result to a comparator so the processor can tell when an operation produces zero. That signal is needed later for status reporting and conditional jump instructions.

What this proves. The test subtracts 9 from 9. The result is 00000000 and the Zero output becomes 1, proving that the zero-detection logic responds correctly.

Step 5: Integrating the 8-Bit ALU

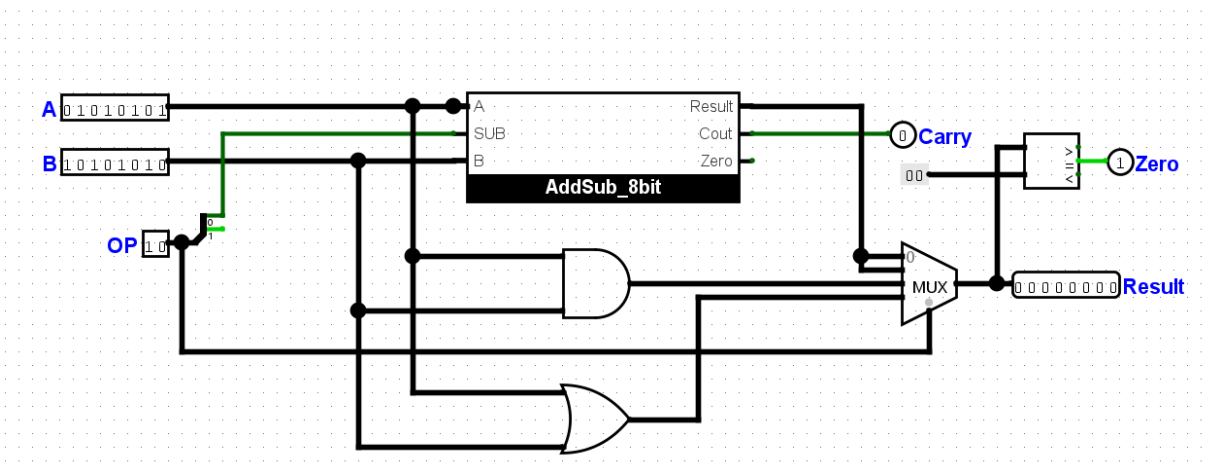


Figure 5. Integrating the 8-Bit ALU

What I built or tested. I combined addition, subtraction, AND, and OR logic behind a four-input multiplexer. The two-bit OP control selects which result is sent to the ALU output.

What this proves. With OP set to 10, the ALU performs AND on 01010101 and 10101010. Because none of the matching bit positions are both 1, the result is 00000000 and Zero becomes 1. This verifies the ALU selection logic and its zero output.

Step 6: Testing the 8-Bit Register Hold Function

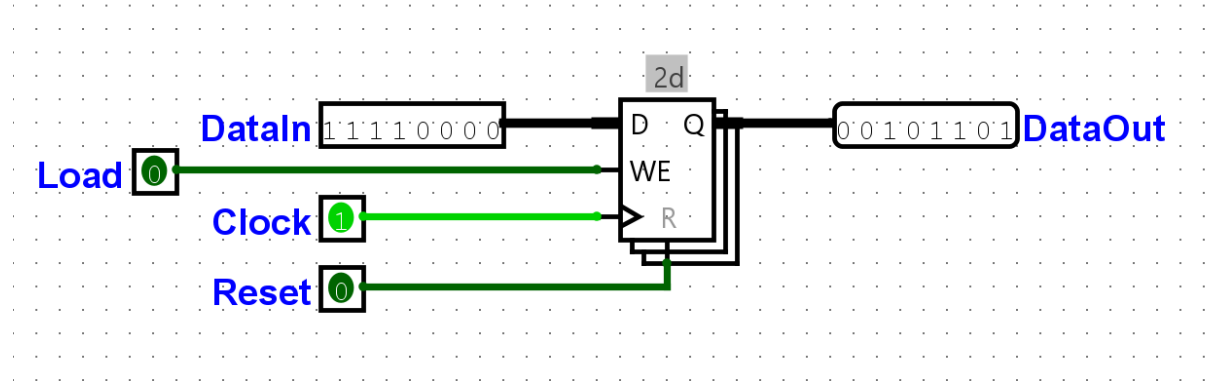


Figure 6. Testing the 8-Bit Register Hold Function

What I built or tested. I built an 8-bit register with DataIn, Load, Clock, Reset, and DataOut connections. The Load signal controls whether a rising clock edge stores new data or keeps the previous value.

What this proves. In this screenshot, DataIn has changed to 11110000, but Load is 0. DataOut stays at its previously stored value, 00101101. This confirms that the register does not overwrite data when loading is disabled.

Step 7: Testing the Control Unit Halt Instruction

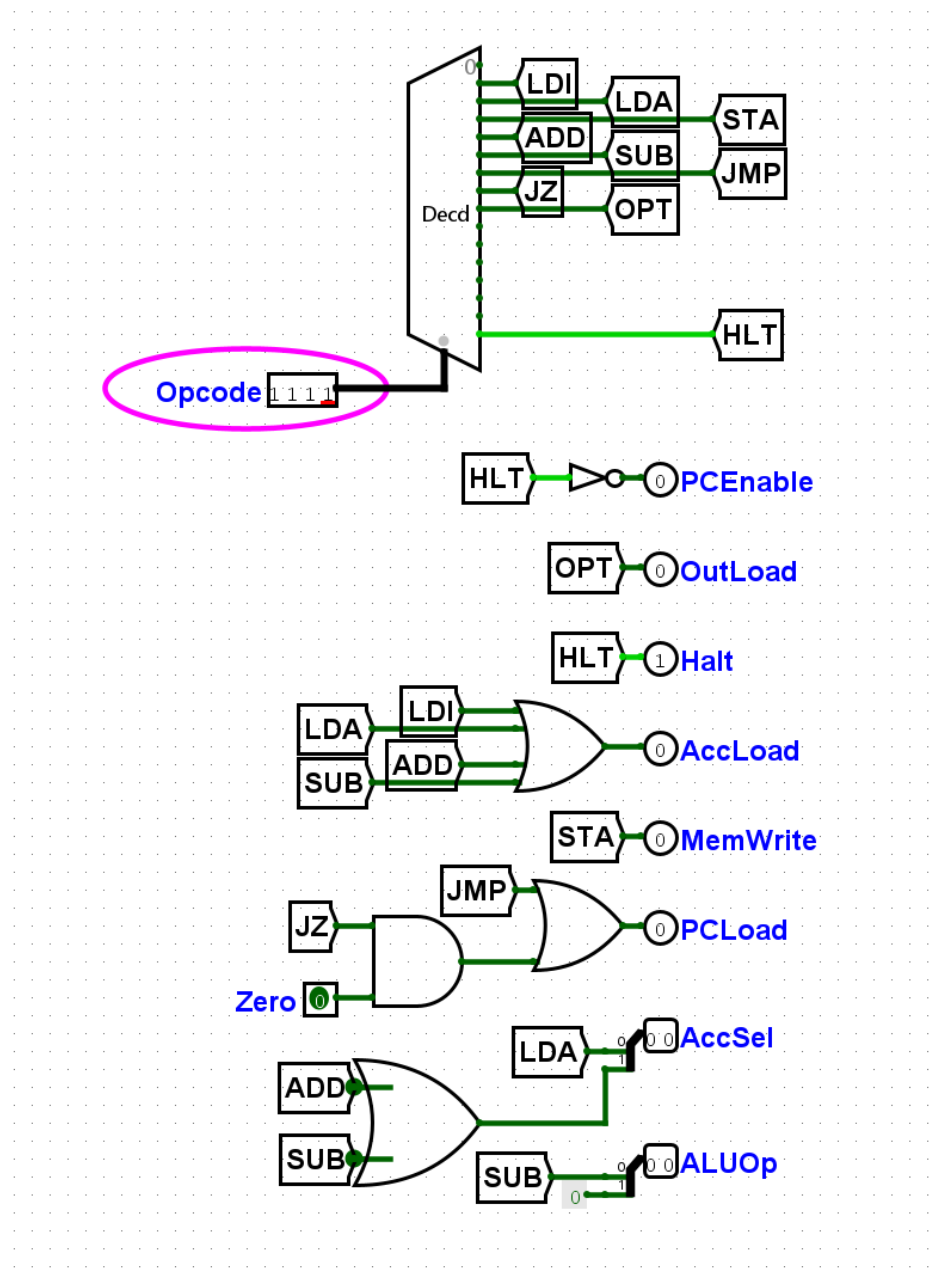


Figure 7. Testing the Control Unit Halt Instruction

What I built or tested. I decoded the 4-bit opcode into individual instruction signals and combined those signals to create the processor controls. Opcode 1111 is assigned to HLT.

What this proves. When HLT is selected, Halt becomes 1 and PCEnable becomes 0 while the other write and load controls remain inactive. This proves the control unit can stop the program counter without causing an unwanted memory or register write.

Step 8: Writing to and Reading from Data Memory

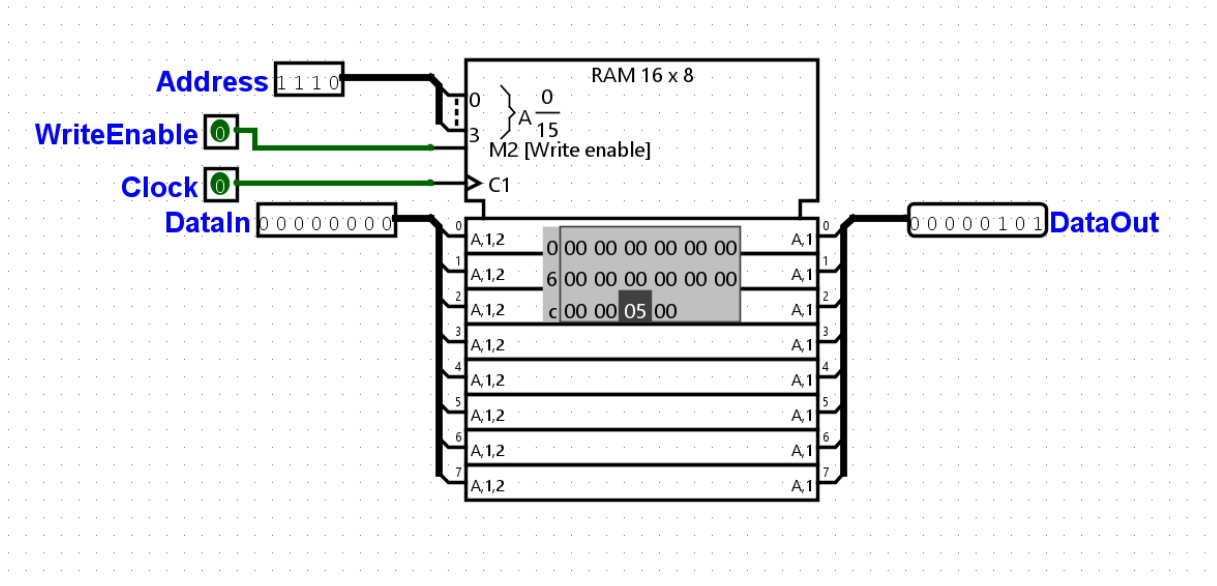


Figure 8. Writing to and Reading from Data Memory

What I built or tested. I configured a 16-address by 8-bit RAM module with a 4-bit address, 8-bit data input, write enable, clock, and asynchronous data output.

What this proves. The value 05 was previously written to address E. With address set to 1110 and write enable turned off, DataOut reads 00000101. This confirms that the RAM stored the value and can read it back without rewriting the location.

Step 9: Resetting the Completed CPU

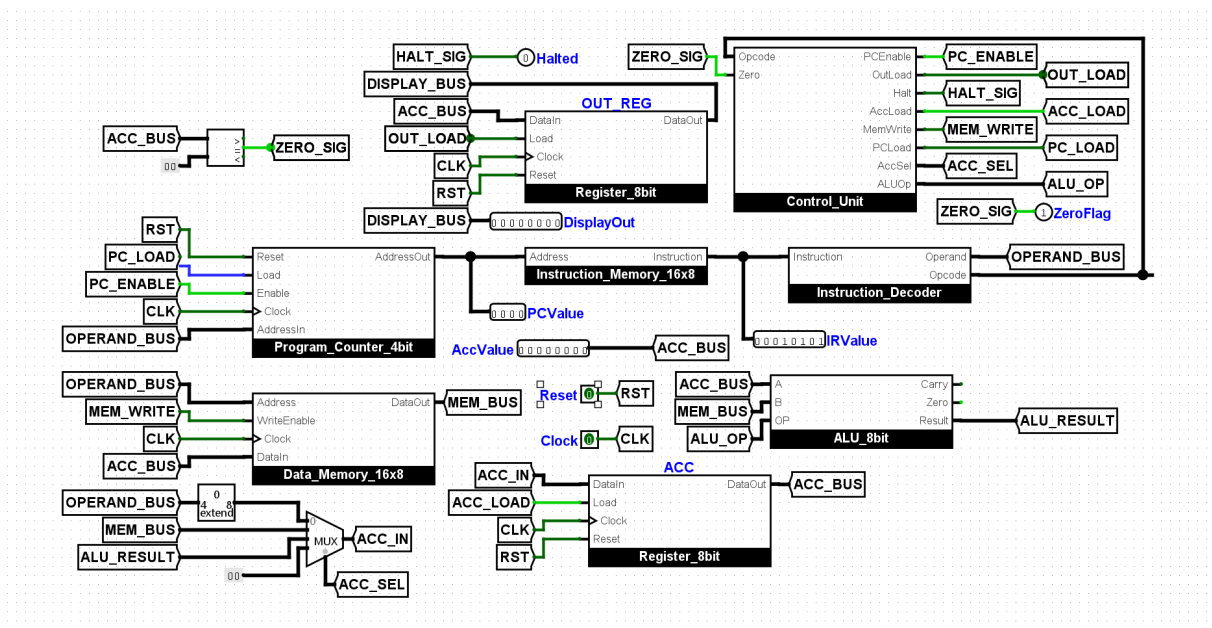


Figure 9. Resetting the Completed CPU

What I built or tested. After testing the individual circuits, I connected the program counter, instruction memory, decoder, control unit, data memory, ALU, accumulator, output register, buses, and status logic into one CPU.

What this proves. The reset sequence places the program counter at 0000, clears the accumulator and display, and makes ZeroFlag equal 1. IRValue shows 00010101, the first instruction in ROM: LDI 5.

Step 12: Program Step 3: Load Immediate 3

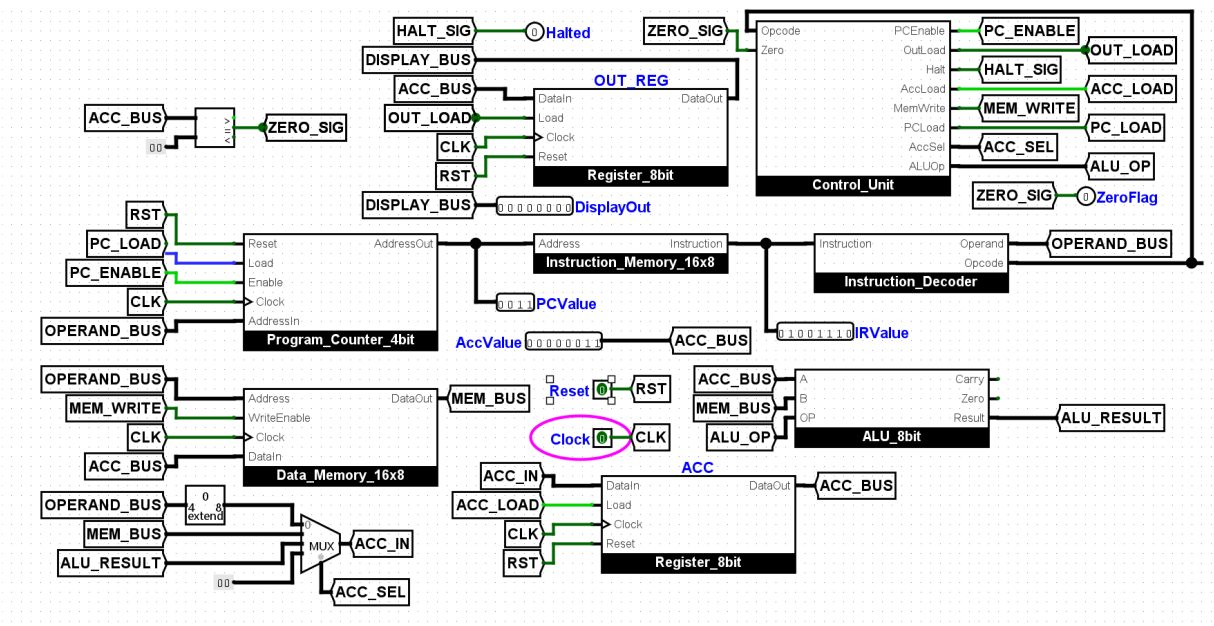


Figure 12. Program Step 3: Load Immediate 3

What I built or tested. The second load-immediate instruction replaces the accumulator value with 3. The accumulator input multiplexer again selects the immediate-data path.

What this proves. After the clock pulse, AccValue is 00000011 and PCValue is 0011. IRValue becomes 01001110, which is ADD E.

Step 13: Program Step 4: Add the Stored Value

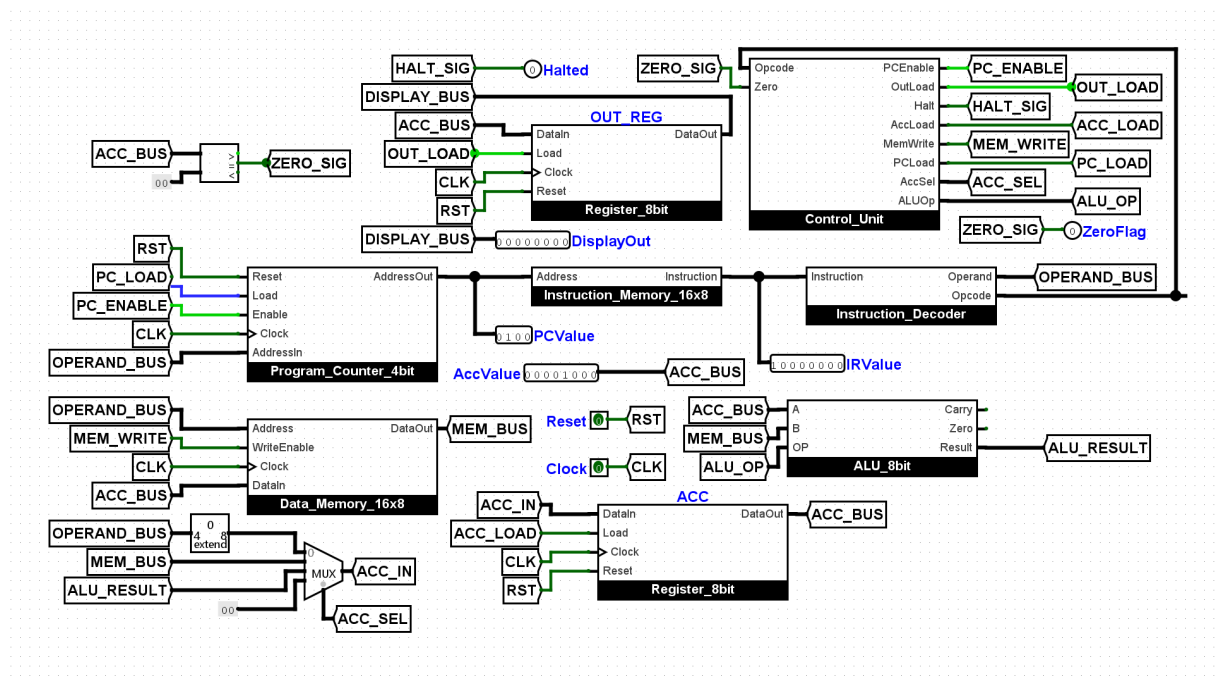


Figure 13. Program Step 4: Add the Stored Value

What I built or tested. ADD E reads the value 5 from RAM address E and sends it to the ALU with the accumulator value 3. The control unit selects addition and enables the accumulator to load the result.

What this proves. AccValue becomes 00001000, showing that the processor correctly calculated $3 + 5 = 8$. PCValue advances to 0100 and IRValue becomes 10000000, the output instruction.

Step 14: Program Steps 5 and 6: Output 8 and Reach Halt

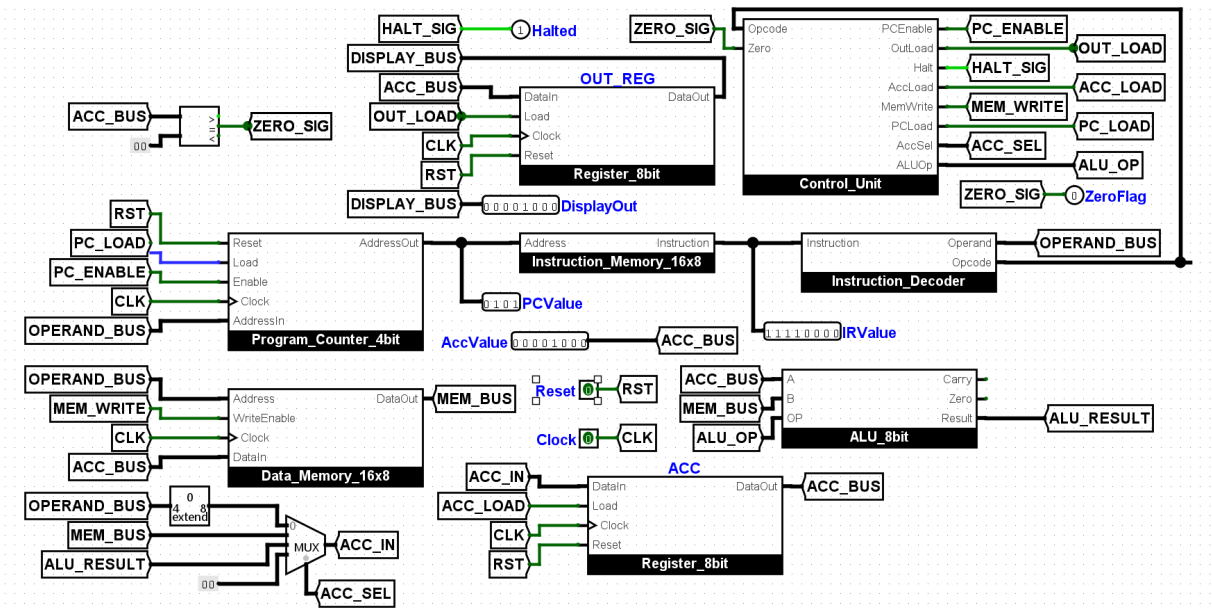


Figure 14. Program Steps 5 and 6: Output 8 and Reach Halt

What I built or tested. The OPT instruction copies the accumulator into the output register. The next ROM location contains HLT, which immediately drives the Halt signal and disables the program counter.

What this proves. DisplayOut and AccValue both show 00001000, PCValue is 0101, IRValue is 11110000, and Halted is 1. This is the expected result of the demonstration program.

Step 15: Final Verification: Halt Holds the Processor State

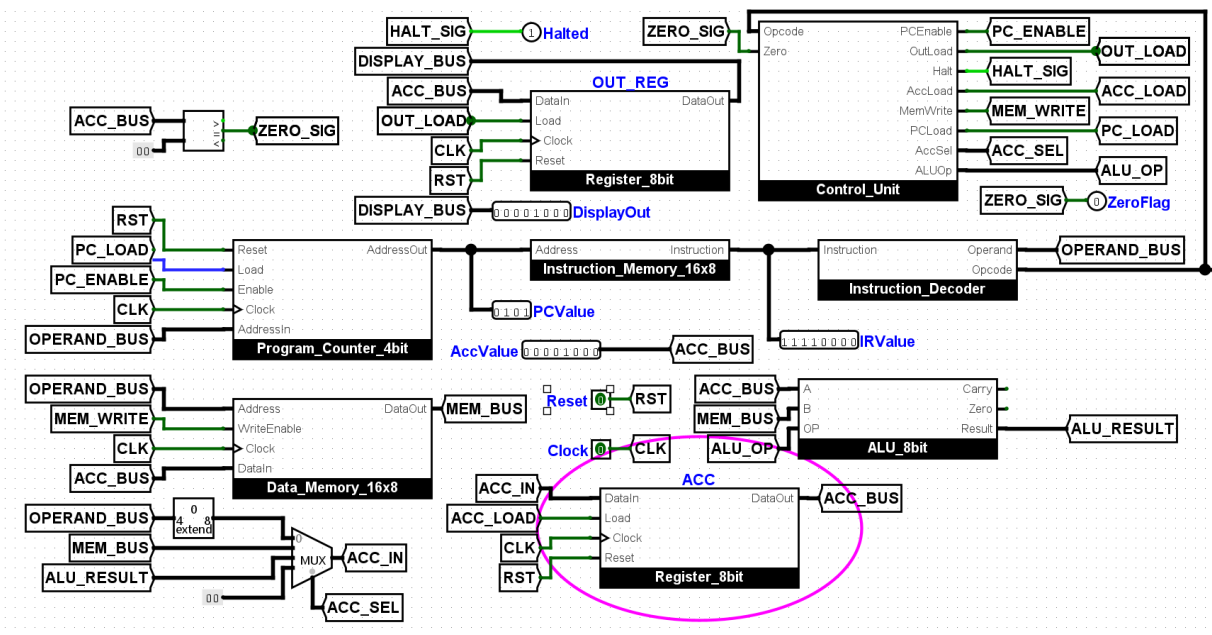


Figure 15. Final Verification: Halt Holds the Processor State

What I built or tested. I applied one additional clock pulse after the processor entered the halt state. This checks that HLT actually freezes program execution instead of only turning on an indicator.

What this proves. PCValue remains 0101, DisplayOut remains 00001000, AccValue remains 00001000, and Halted remains 1. The unchanged state confirms that PCEnable is disabled and the processor is holding its completed result.

Final Result and Reflection

The completed processor successfully ran the full demonstration program. It loaded 5, stored it in RAM, loaded 3, added stored 5, placed the answer 8 on the display output, and halted at the correct instruction. A second clock pulse after HLT did not change the program counter or output, which confirmed that the halt logic worked instead of only lighting an indicator.

The most useful part of this project was building the processor one system at a time. When something showed an undefined value or an incompatible-width error, I could trace it back to a specific tunnel, bus width, register output, or control connection. That made the troubleshooting part of the design process instead of guessing at the whole CPU at once.

This project demonstrates that I can design, integrate, test, and explain a functional digital processor.